

Back-end em python



Prof^a. Adrielly Sayonara

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 19 - Revisão de POO + Tratamento de Exceções

Objetivos da Aula

- Reforçar os principais conceitos de Programação Orientada a Objetos (POO) em Python
- Introduzir o tratamento de erros com try/except
- Aplicar conceitos em pequenos scripts e sistemas
- Preparar base para trabalhar com APIs e sistemas robustos





Aula 19 - Revisão de POO + Tratamento de Exceções

Principais conceitos da POO

- Classe e objeto
- Encapsulamento
- Herança
- Polimorfismo





Aula 19 - Revisão de POO + Tratamento de Exceções



Exemplo simples de classe

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco
```

- `__init__` é o construtor
- `self` representa o objeto atual



Aula 19 - Revisão de POO + Tratamento de Exceções



Adicionando métodos à classe

```
p1 = Produto("Tênis", 199.90)
p1.exibir_info()
```

- Criamos objetos baseados na classe
- Cada objeto tem seus próprios dados



Aula 19 - Revisão de POO + Tratamento de Exceções



Adicionando métodos à classe

```
class Produto:
    def __init__(self, nome, preco):
        self.nome = nome
        self.preco = preco

    def exibir_info(self):
        print(f"Produto: {self.nome} | Preço: R${self.preco}")
```



Aula 19 - Revisão de POO + Tratamento de Exceções



Instanciando objetos

```
p1 = Produto("Tênis", 199.90)
p1.exibir_info()
```

- Criamos objetos baseados na classe
- Cada objetos tem seus próprios dados



Aula 19 - Revisão de POO + Tratamento de Exceções

Herança em Python

```
class Eletronico(Produto):  
    def __init__(self, nome, preco, voltagem):  
        super().__init__(nome, preco)  
        self.voltagem = voltagem
```

- super() chama o construtor da classe pai
- Reutilização de código entre classes relacionadas





Aula 19 - Revisão de POO + Tratamento de Exceções



Herança em Python

```
class Produto:
    def aplicar_desconto(self):
        pass

class Livro(Produto):
    def aplicar_desconto(self):
        print("10% de desconto aplicado!")
```

- Cada classe pode redefinir comportamentos conforme sua lógica



Aula 19 - Revisão de POO + Tratamento de Exceções



Herança em Python

- Erros acontecem! O tratamento evita falhas no programa
- Exemplo:

```
numero = int(input("Digite um número: "))
```

- Se digitar texto, o programa quebra



Aula 19 - Revisão de POO + Tratamento de Exceções



Introdução ao try/except

```
try:
    numero = int(input("Digite um número: "))
    print(10 / numero)
except ValueError:
    print("Você deve digitar um número inteiro!")
except ZeroDivisionError:
    print("Não é possível dividir por zero!")
```



Aula 19 - Revisão de POO + Tratamento de Exceções

Estrutura básica do try/except

- **try**: onde o erro pode ocorrer
- **except**: onde tratamos o erro
- **else**: executa se não houver erro
- **finally**: executa sempre (opcional)





Aula 19 - Revisão de POO + Tratamento de Exceções

Exemplo com else e finally

```
try:
    arquivo = open("dados.txt")
    conteudo = arquivo.read()
except FileNotFoundError:
    print("Arquivo não encontrado.")
else:
    print(conteudo)
finally:
    print("Processo encerrado.")
```





Aula 19 - Revisão de POO + Tratamento de Exceções

Atividade 1 - Tratamento de Erros de Entrada

Criar um programa que:

- Solicita dois números
- Realiza a divisão
- Trata divisão por zero e entradas inválidas





Aula 19 - Revisão de POO + Tratamento de Exceções



Atividade 2 - Classe com tratamento interno

Criar uma classe **Pessoa** com:

- Método para definir idade via input
- Tratamento de erros com try/except dentro do método



Aula 19 - Revisão de POO + Tratamento de Exceções



Atividade 3 - Cadastro de produtos com validação

Criar uma classe **Produto** com:

- Método **cadastrar()** que pede nome e preço
- Valida o preço com try/except (float e valor positivo)



Aula 19 - Revisão de POO + Tratamento de Exceções

Agora é sua vez!

- Apresentem suas soluções
- Quais melhorias podemos fazer?





Aula 19 - Revisão de POO + Tratamento de Exceções

Boas práticas com try/except

- Nunca usar `except`: genérico
- Especifique o tipo de exceção
- Evite capturar exceções e não fazer nada
- Mantenha o código limpo e previsível





Aula 19 - Revisão de POO + Tratamento de Exceções

- Revisamos os pilares da POO em Python
- Aprendemos a tratar erros com try/except
- Usamos POO + exceções em exercícios práticos
- Próxima aula: Testes de unidade com métodos OO





Aula 19 - Revisão de POO + Tratamento de Exceções



Materiais e Referências Online

- Curso POO Python – Curso em Vídeo:
<https://www.youtube.com/@CursoemVideo>
- Documentação oficial do Python:
<https://docs.python.org/pt-br/3/tutorial/errors.html>
- Livro: Use a Cabeça! Programação Orientada a Objetos
- PythonBrasil + comunidades: <https://python.org.br/comunidade>



Aula 09 – Branches, Merge e Conflitos com Git + GitHub



Alguma dúvida ?

prof.adrielly.sayonara@gmail.com



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO